

Name: Ayushi Srivastava
SRN No.: PES1UG24CS105
Semester: V
Section: B
Course Code: UE24CS341A
Course Name: Software Engineering

LAB 3 – Architectural Analysis and Component Design

Link: <https://github.com/AyushiSrivastava7/SE-Labs-PES1UG24CS105/tree/main/Lab>

1. Comparison of Architectural Styles

Before selecting an architecture for the **Podcast Guest Scheduling & Outline Builder**, three common architectural styles were considered: Layered, Microservices, and Client-Server.

Architecture	Structure	Advantages	Disadvantages	Suitability
Layered Architecture	Presentation → Business → Data	Simple to understand, well organized, and easy to maintain	Components can become tightly connected, and scaling individual parts can be difficult	Good
Microservices Architecture	Independent services such as Booking, Notification, and Outline Generation	Each service can be developed, deployed, and scaled independently. Failure in one service is less likely to affect the whole system	More complex to develop and maintain, and communication between services can cause network delays	Best
Client-Server Architecture	Clients communicate with a central server	Simple structure, centralized control, and easier data management	The central server can become a bottleneck or single point of failure	Moderate

Selected Architecture

After comparing the three options, **Microservices Architecture** was selected for this system.

The main reason is that the system has several different functions, such as scheduling bookings, managing users, generating episode outlines, managing episodes, and sending notifications. These functions can be separated into individual services and can work together when required.

2. Architecture Selection – Microservices

Why Microservices?

The **Podcast Guest Scheduling & Outline Builder** has multiple functions that can be separated into independent services. This makes Microservices a suitable architecture for the system.

Reason 1 – Independent Services

Services such as the **Booking Service, Outline Generator, and Notification Service** can work independently.

For example, generating an episode outline does not need to directly modify the booking information. This separation makes the system easier to develop and maintain.

Reason 2 – Better Scalability

Different services may have different levels of usage. For example, the Outline Generator may receive many requests when several episodes are being prepared.

With Microservices, the Outline Generator can be scaled separately without having to scale the entire application.

Security Advantage

Each service can have its own authentication and authorization checks. This helps ensure that only authorized users, such as podcast hosts and guests, can access sensitive information such as bookings, guest details, and episode outlines.

Performance Advantage

The Outline Generation Service can handle run-of-show and PDF generation separately from the other services. This allows resources to be focused on the service when there are many requests and helps the system meet the required performance target.

3. Component Identification

The following five components are identified for the system.

1. Booking and Scheduling Service

This service manages everything related to podcast bookings and schedules.

It handles:

- Available time slots
- Booking requests
- Blocked dates
- Approval and rejection of bookings
- Rescheduling

2. User Management Service

This service manages the users of the system.

It handles:

- Podcast guests
- Show hosts
- Authentication
- Authorization
- Guest information

3. Outline Generator Service

This service is responsible for creating the episode outline and run-of-show.

It handles:

- Guest discussion topics
- Timestamp generation
- Run-of-show creation
- PDF production sheets

4. Episode Management Service

This service manages the information related to podcast episodes.

It handles:

- Episode boards
- Approved discussion topics
- Episode schedules
- Editing of the run-of-show by the host

5. Notification Service

This service sends notifications to guests and hosts.

It handles:

- Booking notifications
- Approval and rejection notifications
- Rescheduling notifications
- Calendar-related notifications

4. Provided and Required Interfaces

The main interfaces are:

Component	Provided Interface	Required Interface
Booking & Scheduling Service	Booking API	User API, Notification API
User Management Service	User/Auth API	—
Outline Generator Service	Outline Generation API	Episode API
Episode Management Service	Episode API	Booking API, Outline API
Notification Service	Notification API	User API

For the UML component diagram, these interfaces can be represented using the standard UML lollipop (**provided interface**) and **socket (required interface)** notation.

5. Recommended Component Connections

Connection 1 – Booking & Scheduling → User Management

Interface: User/Auth API

Purpose:

Used to verify the identity and permissions of the guest or host.

Connection 2 – Episode Management → Booking & Scheduling

Interface: Booking API

Purpose:

Used to obtain approved booking and scheduling information.

Connection 3 – Episode Management → Outline Generator

Interface: Outline Generation API

Purpose:

Used to generate the timestamped run-of-show for an episode.

Connection 4 – Booking & Scheduling → Notification

Interface: Notification API

Purpose:

Used to notify guests and hosts when the booking status changes.

Connection 5 – Outline Generator → Episode Management

Interface: Episode API

Purpose:

Used to obtain approved discussion topics and episode scheduling information.

6. UML Component Diagram

